

```

# Inside your index_worker.py or where ProjectLoadWorker is defined
import os
import sqlite3
from PySide6.QtCore import QObject, Signal, Slot
from pathlib import Path

from LatexIndexParser import LatexIndexParser

class ProjectLoadWorker(QObject):
    statusUpdated = Signal(str)
    finished = Signal(bool, list, list, list) # (is_db_restored, headings, references, file_tree)
    errorOccurred = Signal(str)

    def __init__(self, db_path: str, project_root: str, tex_file_paths: list):
        super().__init__()
        # Convert strings to fully resolved, absolute pathlib Objects
        # This completely strips out double slashes, relative dots, and casing discrepancies
        self.db_path = Path(db_path).resolve()
        self.project_root = Path(project_root).resolve()
        self.tex_file_paths = tex_file_paths

    @Slot()
    def process(self):
        try:
            # 1. Map the left-hand directory file tree
            self.statusUpdated.emit("Scanning project subdirectory structures...")
            file_tree_payload = []
            self._scan_folder_data(str(self.project_root), file_tree_payload)

            # --- ROBUST PATHLIB EXISTENCE VALIDATION ---
            actual_db_to_load = None

            # Use pathlib's absolute evaluation engine
            if self.db_path.exists() and self.db_path.is_file():
                actual_db_to_load = self.db_path
                print(f"DEBUG WORKER: Target database localized via path matching: {actual_db_to_load}")
            else:
                print(f"DEBUG WORKER: Direct path not found on disk: {self.db_path}. Scanning directory elements...")
                # Fallback: scan directory iteratively using pathlib glob to discover any valid DB footprint
                for file_item in self.project_root.iterdir():
                    if file_item.is_file() and file_item.name.lower().endswith("_index data.db"):
                        actual_db_to_load = file_item
                        # Re-sync string representation back to tracking registers
                        self.db_path = file_item
                        print(f"DEBUG WORKER: Deterministic scan discovered database: {actual_db_to_load}")
                        break

            # =====
            # Scenario A: Verifiable database asset is detected on disk. Load from tables.
            # =====
            if actual_db_to_load:
                self.statusUpdated.emit("Database found. Loading structural indexes from persistence...")

                # Pass str(path) since sqlite3 expects a clean string reference pattern
                conn = sqlite3.connect(str(actual_db_to_load))
                conn.row_factory = sqlite3.Row
                cursor = conn.cursor()

                headings = [dict(r) for r in cursor.execute("SELECT * FROM index_headings").fetchall()]
                references = [dict(r) for r in cursor.execute("SELECT * FROM index_references").fetchall()]
                conn.close()

                # Emit result: True strictly shortcuts the regex scraper step
                self.finished.emit(True, headings, references, file_tree_payload)
                return

            # =====
            # Scenario B: Pure empty project folder fallback loop. Run regular expression sweep.
            # =====
            self.statusUpdated.emit("Harvesting index macros via regex fallback engine...")
            headings_payload = []
            references_payload = []

            # Use an internal tracking lookup dictionary to avoid duplicate headings text nodes
            seen_headings = {}
            running_id_pool = 1
            heading_id_counter = 1

            for file_path in self.tex_file_paths:
                norm_target = str(Path(file_path).resolve())
                filename = Path(norm_target).name
                self.statusUpdated.emit(f"Parsing index definitions: {filename}")

                payloads, next_id = LatexIndexParser.parse_file(norm_target, start_id=running_id_pool)
                running_id_pool = next_id

```

```

for parts_list, uid_dict in payloads:
    # Generate uniform '!' path syntax string representations
    full_heading_path = "!".join(parts_list)
    path_key = full_heading_path.lower().strip()

    # Deduplicate nodes across files using an asset map registry
    if path_key not in seen_headings:
        assigned_heading_id = heading_id_counter
        seen_headings[path_key] = assigned_heading_id

        headings_payload.append({
            "id": assigned_heading_id,
            "parent_id": None,
            "heading_text": full_heading_path,
            "name": full_heading_path,
            "depth": len(parts_list) - 1
        })
        heading_id_counter += 1
    else:
        # Re-use the master identifier to merge entries to a single visual layout branch row
        assigned_heading_id = seen_headings[path_key]

    # CRITICAL FIX: Extract coordinates safely using dict.get() with absolute fallbacks
    # This prevents KeyError crashes regardless of whether the parser returns old or new keys
    line_coord = uid_dict.get("line_number") or uid_dict.get("line") or 1
    col_coord = uid_dict.get("column_offset") or uid_dict.get("col") or 0
    file_target = uid_dict.get("file_path") or uid_dict.get("path") or norm_target
    unique_id = uid_dict.get("id") or uid_dict.get("unique_id_number") or running_id_pool

    references_payload.append({
        "heading_id": assigned_heading_id,
        "heading_raw_text": full_heading_path,
        "uid": f"{file_target}:{line_coord}:{col_coord}",
        "unique_id_number": int(unique_id),
        "file_path": str(file_target),
        "line_number": int(line_coord),
        "column_offset": int(col_coord),
        "encap": uid_dict.get("encap", "standard")
    })

    self.finished.emit(False, headings_payload, references_payload, file_tree_payload)

except Exception as e:
    self.errorOccurred.emit(str(e))

def _scan_folder_data(self, current_path: str, output_list: list):
    """Scans folder structure via basic directory queries safely out-of-thread."""
    import os
    try:
        for entry in os.scandir(current_path):
            if entry.name == ".session_backups" or entry.name.lower().endswith(".db"):
                continue
            node_data = {
                "name": entry.name,
                "is_dir": entry.is_dir(),
                "path": entry.path,
                "children": []
            }
            output_list.append(node_data)
            if entry.is_dir():
                self._scan_folder_data(entry.path, node_data["children"])
    except PermissionError:
        pass

```